

Create a *Python Virtual Environment* and install the modules **tensorflow** & **utlralytics** with UV

► Interest

The state of the art in Python programming (Data processing, Machine Learning...) is to develop each projects within a **Python Virtual Environment** (PVE) which provides a dedicated and persistent environment containing a specific installation of Python:

- independent of other Python installations likely to coexist on the same machine,
- independent of computer updates.

A PVE is based on a dedicated disk tree that houses the desired version of the Python interpreter and the specific versions of the Python modules needed for the project. One can create, duplicate, delete and recreate a PVE very easily, without impacting other Python installations possibly present on your computer.

► Tools

The most often used tools until now to create a PVE are:

- the conda command, available if you have installed [miniconda](#) or [Anaconda](#) on your computer
- the venv Python module (see [venv](#))
- the poetry tool (see [poetry.org](#))...

Recently a new tool has been unveiled to manage Python project within a PVE : [uv](#). It is faster and easier to use than the previous tools.

► How to install **uv** on your computer

► 1 – Windows

Type the word **powershell** in the *Windows search bar* and clic on  **Windows PowerShell** to run a **Windows PowerShell**, then copy/paste the command bellow in the PowerShell window and hit ENTER:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

► 2 – Mac OS & GNU/Linux

Open a **terminal window**, then copy/paste the command bellow and hit ENTER:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

If you don't have `curl` installed on your system, then you can use `wget` as shown below:

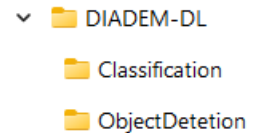
```
wget -qO- https://astral.sh/uv/install.sh | sh
```

► Checking Python modules installation for Deep Learning

This section aims to test if the **tensorflow** & **ultralytics** modules install correctly on your computer.

► 1 – Create the working tree

With the *file manager* of your computer create a new directory named **DIADEM-DL**, enter **DIADEM-DL** and create the 2 new directories **Classification** and **ObjectDetection**:



► 2 – Create the PVE

- With your file manager, copy the path of the **DIADEM-DL** directory.
- Open a new console (Windows: a *powershell* window, Mac/Linux: a *terminal*), and go to the **DIADEM-DL** directory with the **cd** (*change directory*) command:

```
cd <path of DIADEM-DL dir>
```

where <path of DIADEM-DL dir> is the path of the **DIADEM-DL** directory obtained by pasting the path you have just copied.

- Create a PVE with Python 3.12 interpreter in the **Classification** and **ObjectDetection** directories type each of these commands followed by the key ENTER:

```
uv init -p 3.12 Classification
uv init -p 3.12 ObjectDetection
```

► 3 – Installing the tensorflow module

Change your current directory to **Classification** with the command:

```
cd Classification
```

Try to install the version 2.17 of tensorflow:

```
uv add tensorflow==2.17
```

► Windows

There is a bug when installing tensorflow under Windows which you must fix with the command:

```
uv pip install tensorflow==2.17
```

Now try to use the tensorflow module to see if it works correctly:

```
uv run python -c "import tensorflow as tf"
```

↪ you should get some warning messages but no errors.

↪ If something wrong occurs, make a snapshot of the problem, and send it to me.

► 4 – Installing the ultralytics module

Change your current directory to the **ObjectDetection** directory with the command:

```
cd ../ObjectDetection
```

Try to install the ultralytics module :

```
uv add ultralytics
```

If the previous command succeeds, try to use the ultralytics module to see if it works correctly:

```
uv run python -c "from ultralytics import YOLO; print(dir(YOLO))"
```

↪ you should get many output lines but no errors.

↪ If something wrong occurs, make a snapshot of the problem, and send it to me.